

TECHNICAL TRAINING COURSE

Overview of Web Services

- HTTP, XML & SOAP Messaging
- WSDL & Service Contracts
- SOA Architecture Principles
- REST vs SOAP Visions
- UDDI, WS-I & Security

2-Day Course | Intermediate–Advanced | Developer / Architect



COURSE AGENDA

10 Topics — 2 Days — Hands-On Labs

01 Why Web Services?

02 Service-Oriented Architecture

03 HTTP and XML

04 SOAP

05 WSDL

06 The SOAP Vision

07 The REST Vision

08 UDDI

09 WS-I Basic Profile

10 Security

Why Web Services?

The integration problem and the standards-based solution

40–70%

of IT budgets consumed by
integration work, not features

- Point-to-point connections scale as $O(n^2)$
- Proprietary protocols (CORBA, DCOM) are firewall-hostile
- EDI/VAN is batch-only and expensive
- Vendor lock-in with every integration

The Web Services Triangle

SOAP

Messaging Protocol
XML over HTTP

WSDL

Service Description
Machine-readable contract

UDDI

Service Discovery
Publish, Find, Bind

HTTP + XML (universally available) · Open Standards (vendor-neutral) · Firewall Friendly (port 80/443)

Service-Oriented Architecture

Design principles for building composable, reusable services

Standardised Contract

Formal interface definitions via WSDL & schemas

Loose Coupling

Consumers depend on the contract, not the implementation

Reusability

Services designed to serve multiple consumers

Abstraction

Only the interface is public; internals are hidden

Composability

Services orchestrated into higher-order processes

Statelessness

No session state retained between calls

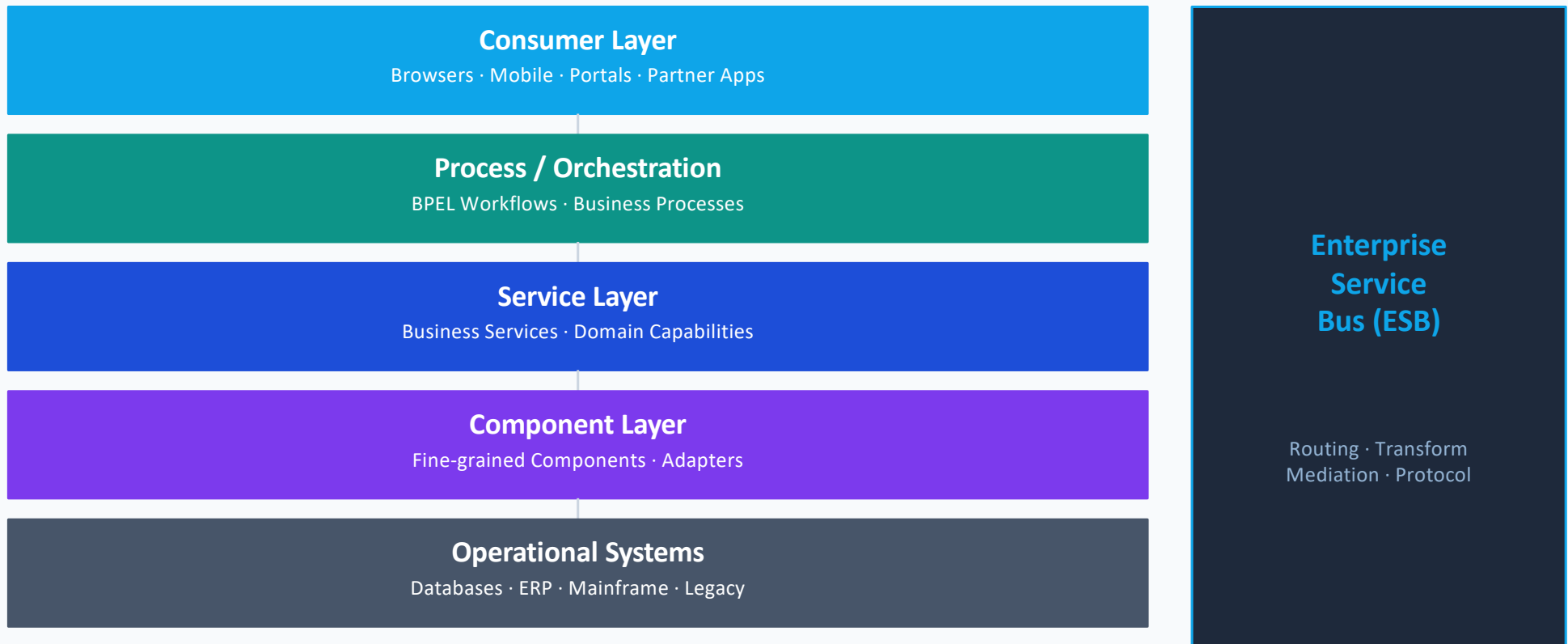
Discoverability

Metadata published so services can be found

Autonomy

Services own and control their own logic

SOA Reference Architecture



HTTP Methods in Web Services

GET

Retrieve a resource. Safe & idempotent.

POST

Create / submit. SOAP uses exclusively.

PUT

Full replacement. Idempotent.

PATCH

Partial update. Not idempotent.

DELETE

Remove resource. Idempotent.

Key Status Codes: 2xx Success · 3xx Redirect · 4xx Client Error · 5xx Server Error

XML — The Data Format

- 1 Single root element
- 2 Every tag must be closed
- 3 Elements properly nested
- 4 Attributes quoted correctly
- 5 Special chars escaped: & < > " '

```
<?xml version="1.0"?>
<order xmlns="http://example.com">
  <orderId>ORD-001</orderId>
  <customer>
    <name>Acme Corp</name>
    <tier>premium</tier>
  </customer>
  <total currency="GBP">
    499.99
  </total>
</order>
```


SOAP

Simple Object Access Protocol

SOAP Envelope

Header (optional)

→ Auth Token, Routing, WS-* extensions

Body

→ Operation / Response / Fault

Transport

HTTP (port 80/443)

Format

XML only

Versions

1.1 (text/xml) and 1.2 (application/soap+xml)

Errors

SOAP Fault element in Body

SOAP 1.2 Message — Request & Response

REQUEST

```
POST /CustomerService HTTP/1.1
Content-Type: application/soap+xml

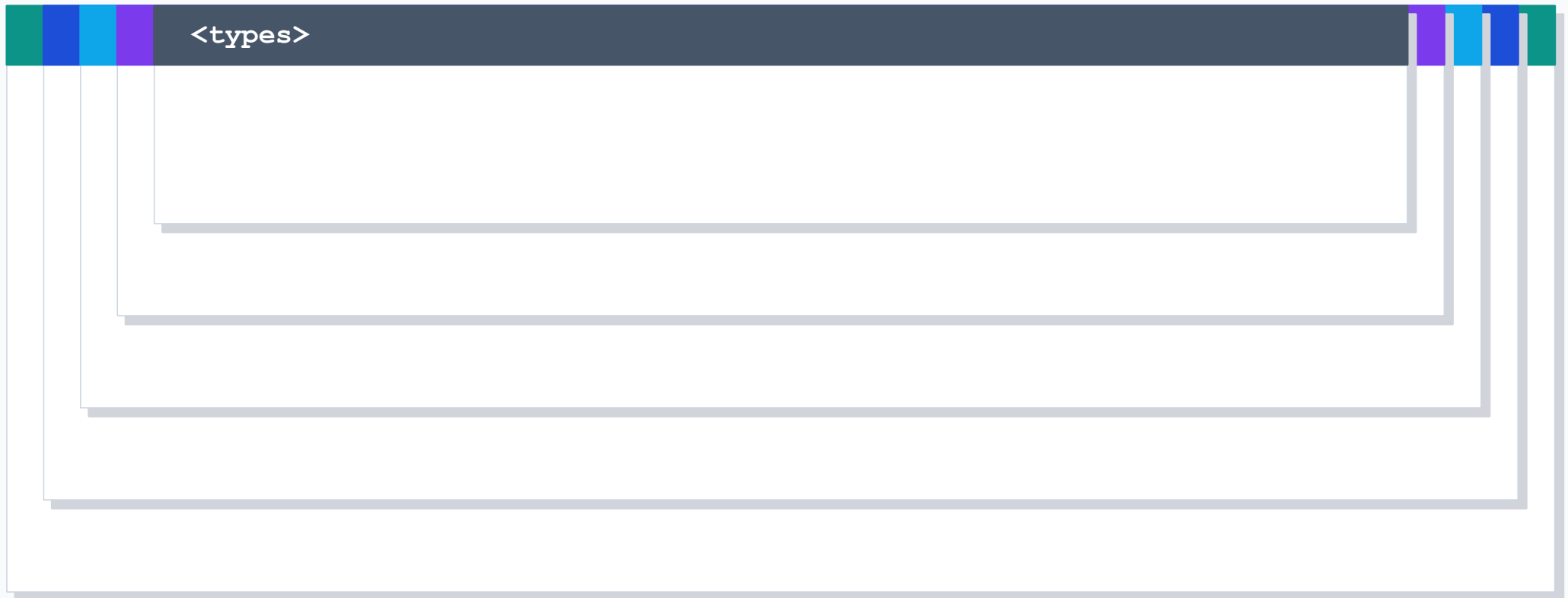
<env:Envelope
  xmlns:env="...soap-envelope"
  xmlns:c="http://example.com/cust">
  <env:Header>
    <c:AuthToken
      env:mustUnderstand="true">
      eyJhbGci...
    </c:AuthToken>
  </env:Header>
  <env:Body>
    <c:GetCustomerRequest>
      <c:id>CUST-001</c:id>
    </c:GetCustomerRequest>
  </env:Body>
</env:Envelope>
```

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/soap+xml

<env:Envelope
  xmlns:env="...soap-envelope">
  <env:Body>
    <c:GetCustomerResponse>
      <c:customer>
        <c:id>CUST-001</c:id>
        <c:name>Acme Corp</c:name>
        <c:tier>PREMIUM</c:tier>
        <c:limit currency="GBP">
          50000
        </c:limit>
      </c:customer>
    </c:GetCustomerResponse>
  </env:Body>
</env:Envelope>
```

WSDL — Web Services Description Language



Analogy: WSDL is the service contract — what you can call, what you send, what you receive, and where to find it.

WSDL Binding Styles — What to Choose

Style	Schema Validate?	WS-I Compliant?	Recommended?
document / literal / wrapped	✓ Yes	✓ Yes	✓ Best choice
document / literal	✓ Yes	✓ Yes	Usable (no wrapper)
rpc / literal	Partial	✓ Yes	Avoid
rpc / encoded	✗ No	✗ No	✗ Avoid (legacy)

✓ Always use document / literal / wrapped for new SOAP services — the only style that is both WS-I compliant and fully schema-validatable.

The SOAP Vision

Enterprise messaging with the WS- extension stack*

WS-Security

Message-level auth, signing & encryption

WS-ReliableMessaging

Guaranteed delivery & ordering

WS-Addressing

Async routing & endpoint references

WS-AtomicTransaction

Distributed 2-phase commit

WS-BusinessActivity

Long-running compensating transactions

WS-Policy

Machine-readable capability declarations

WS-Federation

Identity federation across domains

WS-Trust

Security Token Service (STS)

The REST Vision

Roy Fielding's 2000 dissertation — the web's own architecture

1

Client–Server

Separation of UI from data

2

Stateless

Each request is self-contained

3

Cacheable

Responses declare cacheability

4

Uniform Interface

Consistent interaction model

5

Layered System

Client can't see intermediaries

6

Code on Demand

Optional: server sends executable

REST — Resource Design & URI Patterns

```
# Collections
GET  /customers
POST /customers

# Items
GET    /customers/{id}
PUT    /customers/{id}
PATCH /customers/{id}
DELETE /customers/{id}

# Sub-resources
GET /customers/{id}/orders
GET /customers/{id}/orders/{oid}
```

Nouns not verbs

/customers not /getCustomers

Plural collections

/orders not /order

Hyphens not underscores

/line-items not /line_items

Lowercase URIs

Always; never mixed case

No file extensions

Use Accept header for format

REST vs SOAP — Side by Side

	REST	SOAP
Protocol	HTTP native	HTTP/SMTP/JMS/etc.
Message Format	JSON, XML, binary	XML only
Contract	OpenAPI / Swagger	WSDL (formal standard)
Transactions	No built-in	WS-AtomicTransaction
Security	TLS + OAuth 2.0 + JWT	WS-Security
Caching	Native HTTP caching	No (POST only)
Best For	APIs, Mobile, Microsvcs	Enterprise B2B, Regulated

UDDI — Service Registry

White Pages

Business contact info
Name · Address · Phone

Yellow Pages

Industry categorisation
NAICS · UNSPSC · Geography

Green Pages

Technical service info
WSDL · Endpoint · Binding

Why Public UDDI Registries Failed (shut down 2006):

Low-quality data · Runtime discovery wasn't used · Security concerns · Complexity · Wrong granularity

WS-I Basic Profile — Interoperability Rules

The Basic Profile constrains SOAP/WSDL to ensure services built on different platforms interoperate reliably.

R1000

SOAP Version

Must use SOAP 1.1 (BP 1.1) or 1.2 (BP 2.0)

R2706

Binding Style

document/literal only; RPC/encoded is non-conformant

R2716

Wrapped Pattern

Single wrapper element around all message parts

R2744

SOAPAction Header

Must be present and match the WSDL operation

R2705

Schema Validity

XSD in WSDL types section must be valid

R1031

Content-Type

SOAP 1.1 = text/xml; SOAP 1.2 = application/soap+xml

Security

Protecting web services from threats



Eavesdropping



Tampering



Replay Attack



XML Injection



XML Bomb / XXE



Identity Spoofing



DoS / DDoS



Schema Poisoning

Security — Two Layers of Protection

Transport-Level (TLS/HTTPS)

- Encrypts all traffic in transit
- Prevents eavesdropping & tampering
- Server/client certificate auth
- Mutual TLS for high-assurance B2B
- TLS 1.3 minimum recommended
- Native HTTP caching support

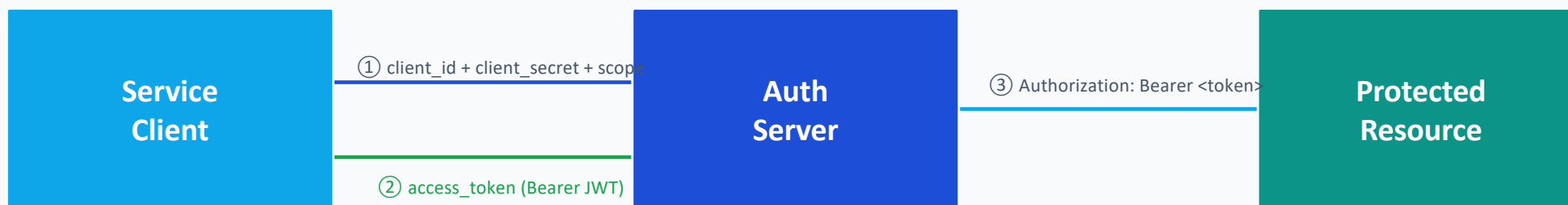
Message-Level (WS-Security)

- Protects individual SOAP elements
- Survives through intermediaries
- UsernameToken · X509 · SAMLToken
- XML Digital Signature (integrity)
- XML Encryption (confidentiality)
- Timestamps prevent replay attacks

Use both layers for SOAP services through intermediaries — TLS secures the wire; WS-Security secures the message.

OAuth 2.0 — Authentication for REST APIs

Client Credentials Flow (Machine-to-Machine)



Client Credentials

Machine-to-machine

Authorization Code

User delegation (web app)

Auth Code + PKCE

SPAs and mobile apps

Refresh Token

Long-lived sessions

XML-Specific Security Vulnerabilities

⚠️ XXE — XML External Entity

Fix: Disable DOCTYPE/DTD processing in all XML parsers

⚠️ XML Bomb (Billion Laughs)

Fix: Limit entity expansion depth; enable secure processing

⚠️ XPath Injection

Fix: Parameterise XPath; never concatenate user input

⚠️ SOAP Array Attack

Fix: Enforce schema validation; limit payload size

⚠️ Schema Poisoning

Fix: Cache schemas locally; verify schema integrity at startup

Key Takeaways

01

Web services solve integration at scale using open, firewall-friendly standards.

02

SOA principles — loose coupling, reusability, composability — guide service design.

03

SOAP + WSDL provides formal, verifiable contracts; ideal for enterprise B2B.

04

REST leverages HTTP natively; right for public APIs, mobile, and microservices.

05

WS-I Basic Profile enforces document/literal/wrapped for guaranteed interoperability.

06

Security requires both TLS (wire) and WS-Security/OAuth (message/identity).

Thank You

Questions & Discussion

Next steps:

- Lab exercises for each section
- End-of-course quiz (30 questions)
- Design exercise: model a web service
- Pass mark: 70% for certification

